

MEMORY INFRASTRUCTURE AT SCALE

STRATUS for hundreds of agents

From one local store to a multi-tenant fleet — what scales, and what to build

Apache-2.0 · github.com/rick-stevens-ai/stratus · Kukla · June 2026

Most of the hard part is done

The bones exist

Four tiers, an HTTP gateway, a background distiller, and lexical+semantic search — all shipping today.

Storage is cheap

One SQLite file + markdown blobs per agent. Hundreds of stores is megabytes, not a cluster.

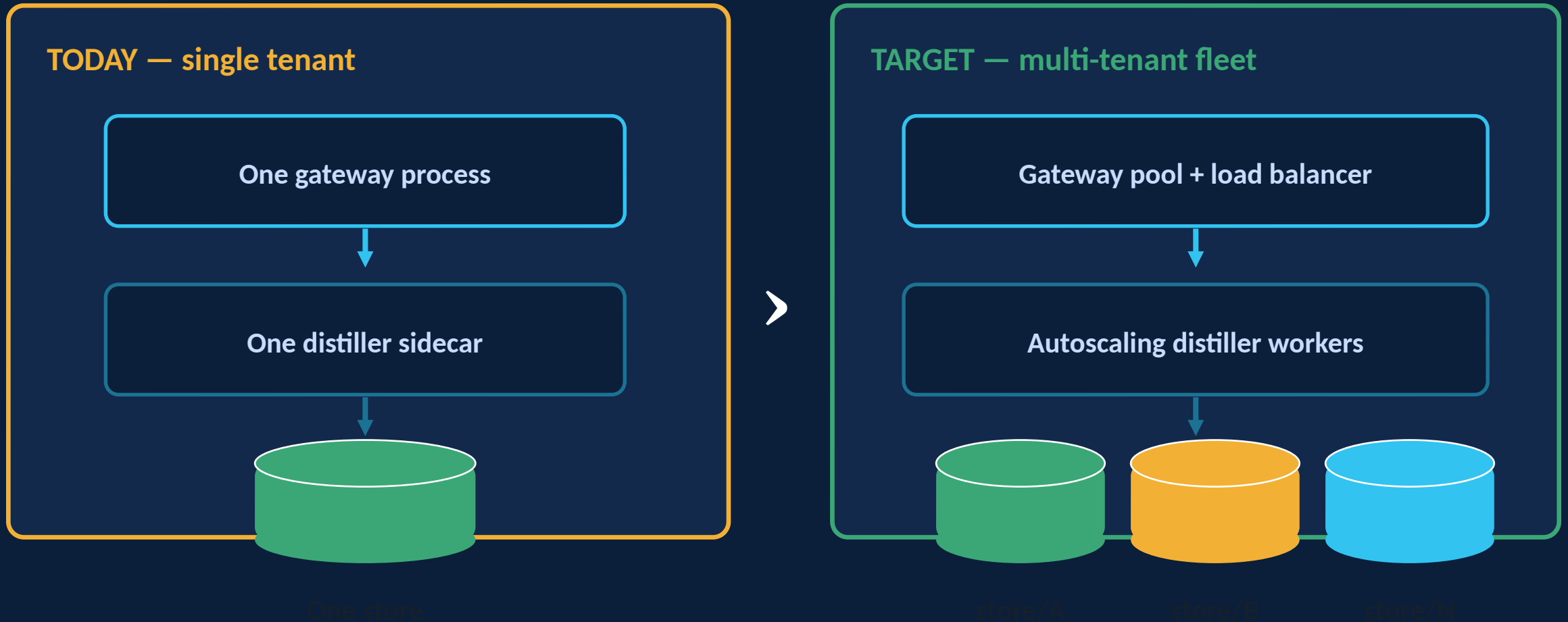
One real gap

Multi-tenant routing — a key that sends each request to its own store. That's the build.

Scaling = add tenant routing + move the distiller to a pool. Not a rewrite.

THE SHIFT

Today → target



What already scales

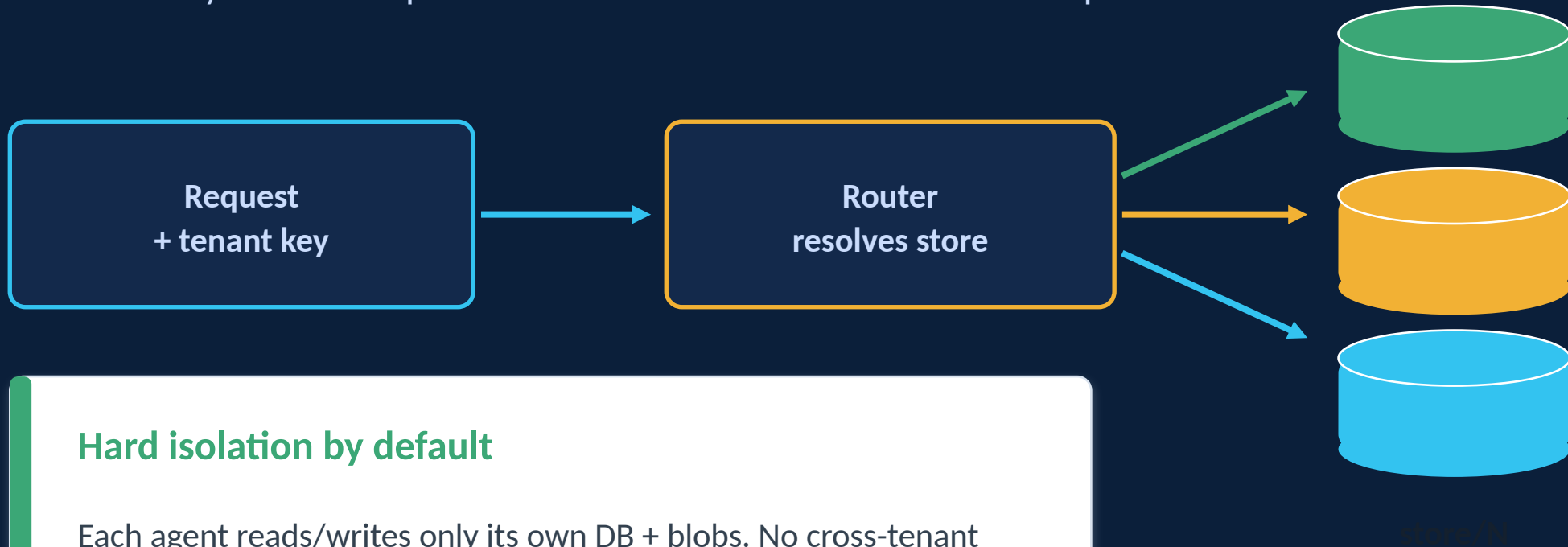
Per-agent store path	DB + blobs already env-driven (STRATUS_DB)	READY
HTTP gateway	Stateless request handling on :8077	READY
Background distiller	Decoupled sidecar; never blocks capture	READY
Tier storage	SQLite + sqlite-vec + markdown — tiny per agent	READY
Tenant routing	One key → one store/blob namespace	TO BUILD

Four of five building blocks already exist. The fifth is a routing layer, not a redesign.

THE ONE NEW PIECE

Tenant routing

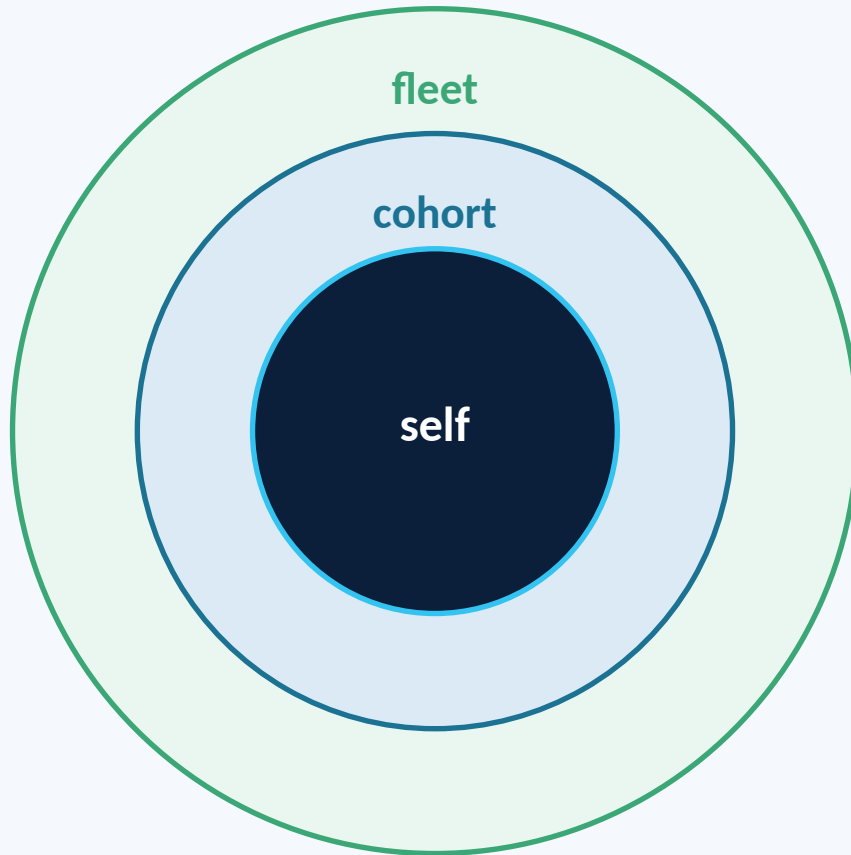
A tenant key on each request selects an isolated store + blob namespace.



Hard isolation by default

Each agent reads/writes only its own DB + blobs. No cross-tenant leakage — the path IS the boundary.

Optional shared recall



self

Only this agent's memory. The private default.

cohort

A named group of agents' shared knowledge.

fleet

Cross-agent search across the whole fleet.

THE COMPUTE CONCERN

Distillation is the real cost

Storage is trivial. The work that scales with agent count is LLM extraction ($T1 \rightarrow T3$).

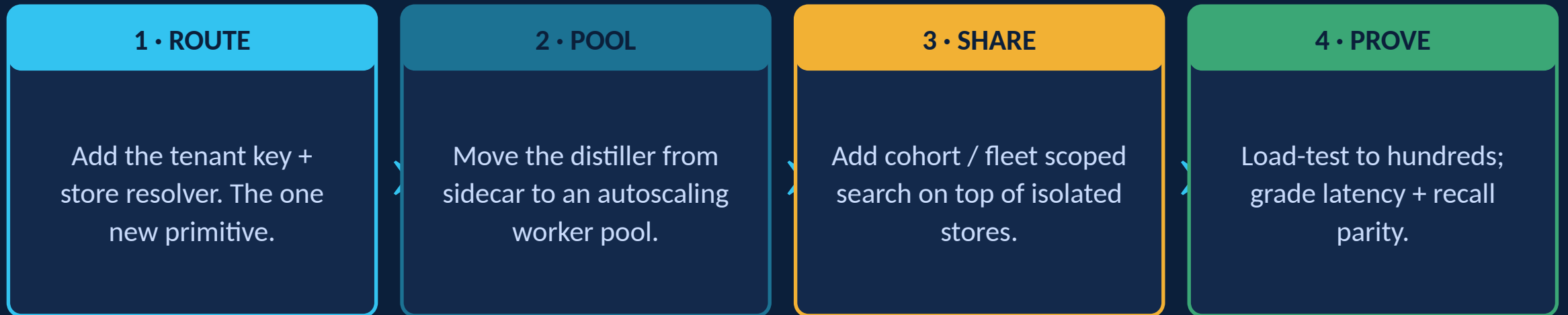


Knobs that hold cost flat

Batch extraction across tenants • throttle distill cadence • size the worker pool to the LLM budget. Capture stays instant regardless — only distillation lags under load, and it lags gracefully.

THE PLAN

Four phases



Phase 1 unblocks everything. Phases 2–4 are independent and can run in parallel.

Where the work is

Tenant routing

Effort MED · Risk LOW. The path IS the isolation boundary — simple and safe.

Distiller pool

Effort MED · Risk LOW. Already decoupled; this is a deployment shape.

Shared-scope search

Effort MED · Risk MED. Cross-tenant queries need careful access rules.

Load to hundreds

Effort MED · Risk MED. LLM throughput is the binding limit, not storage.

The one thing to watch: cross-tenant (fleet) search access rules. Everything else is mechanical.

The shadow run is the harness

What shadow mode bought us

A live week of real traffic captured into STRATUS with zero risk to the agent — plus a weekly compare job that grades capture parity, tier counts, and recall A/B.

Why that matters for scale

The same compare harness becomes the load-test oracle: spin up N synthetic tenants, replay traffic, and grade parity + latency against the single-tenant baseline before any production cutover.

BOTTOM LINE

Five facts



Four-tier engine + gateway + distiller already ship



Storage is trivial — one small store per agent



The only new primitive is tenant routing



The real scaling cost is LLM distillation, and it lags gracefully



The shadow-mode compare harness doubles as the load-test oracle

THE ASK

Build tenant routing first — then scale out the distiller.

One primitive unblocks hundreds of agents. The rest is deployment, and the test harness already exists.